

STUDENT EXPENSE TRACKER

Project Report

Project Type: Python Console Application

Technology: Python 3, JSON, Text File Storage

Student Name	Pushkar Rungta
Registration No.	26BAI10707
Course / Section	BTECH CSE AI/ML
Faculty	RAMRAJ DANGI
Institution	VIT Bhopal University
Academic Year	2026–27

A simple command-line application designed to help students record, search, summarize and manage their expenses while monitoring a monthly budget.

2. Introduction

Managing small daily expenses can become difficult when payments are made for food, travel, outings and other activities. The Student Expense Tracker was developed as a simple Python-based solution for maintaining these records in one place. The application works through a menu-driven command-line interface and stores data locally so that previously added expenses are available when the program is started again.

The project focuses on core Python programming concepts such as functions, lists, dictionaries, loops, conditional statements, exception handling, file handling and the datetime module. JSON is used to store expense records, while a text file stores the monthly budget.

3. Problem Statement

Students often make several small payments during a month, such as food, travel and other personal expenses. Without a simple tracking system, it can be difficult to know how much has been spent, where the money is going, and whether the monthly budget has been crossed.

Problem: Develop a simple and easy-to-use expense tracking system that allows a student to record expenses, view stored records, search by category, calculate spending summaries, set a monthly budget, check the remaining budget and delete incorrect records.

4. Functional Requirements

- **Add Expense:** Accept amount, category and description and automatically record the current date.
- **View Expenses:** Display all saved expenses with their amount, category, description and date.
- **Search Expense:** Search stored expenses by category.
- **Expense Summary:** Calculate total spending and category-wise spending.
- **Set Monthly Budget:** Save a monthly budget in a local file.

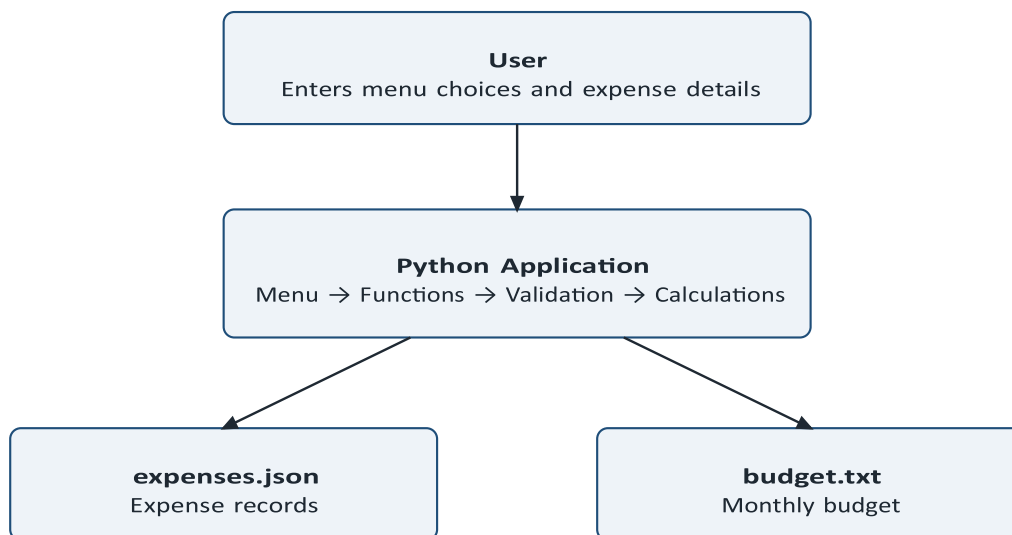
- **Check Budget:** Compare total spending with the saved budget and show the remaining amount.
- **Delete Expense:** Remove a selected expense from the list and save the updated data.
- **Exit:** Close the application from the menu.

5. Non-functional Requirements

- **Usability:** The menu and prompts are simple enough for a beginner to understand.
- **Reliability:** Data is saved after add/delete operations so changes are not lost when the program exits.
- **Maintainability:** Separate functions are used for different operations, making the code easier to modify.
- **Portability:** The project requires only Python 3 and standard-library modules.
- **Performance:** The operations are lightweight and suitable for a small personal expense dataset.
- **Data persistence:** JSON and text files provide basic local persistence without requiring a database server.

6. System Architecture

The application follows a simple local three-part flow: the user interacts with the menu, the Python application processes the selected operation, and local files provide persistent storage.

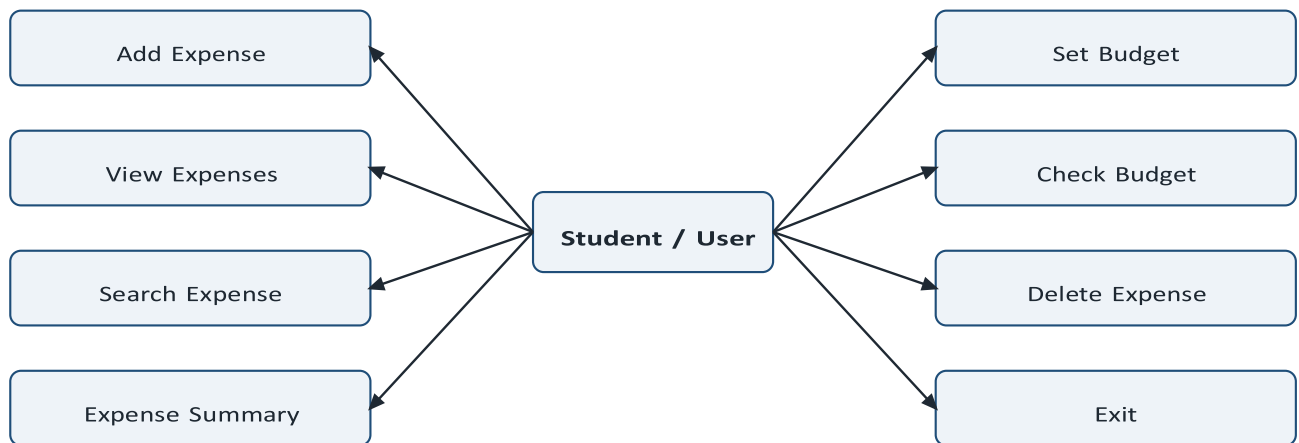


The application starts by loading existing expenses. Menu choices then call dedicated functions. Expense records are stored as dictionaries inside a list and written to JSON. The budget is read from or written to a separate text file.

7. Design Diagrams

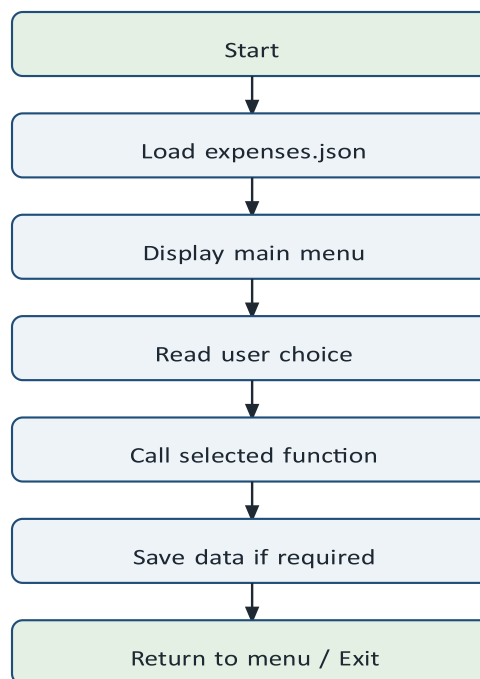
The following diagrams describe the logical design of the application based directly on the implemented Python functions.

7.1 Use Case Diagram

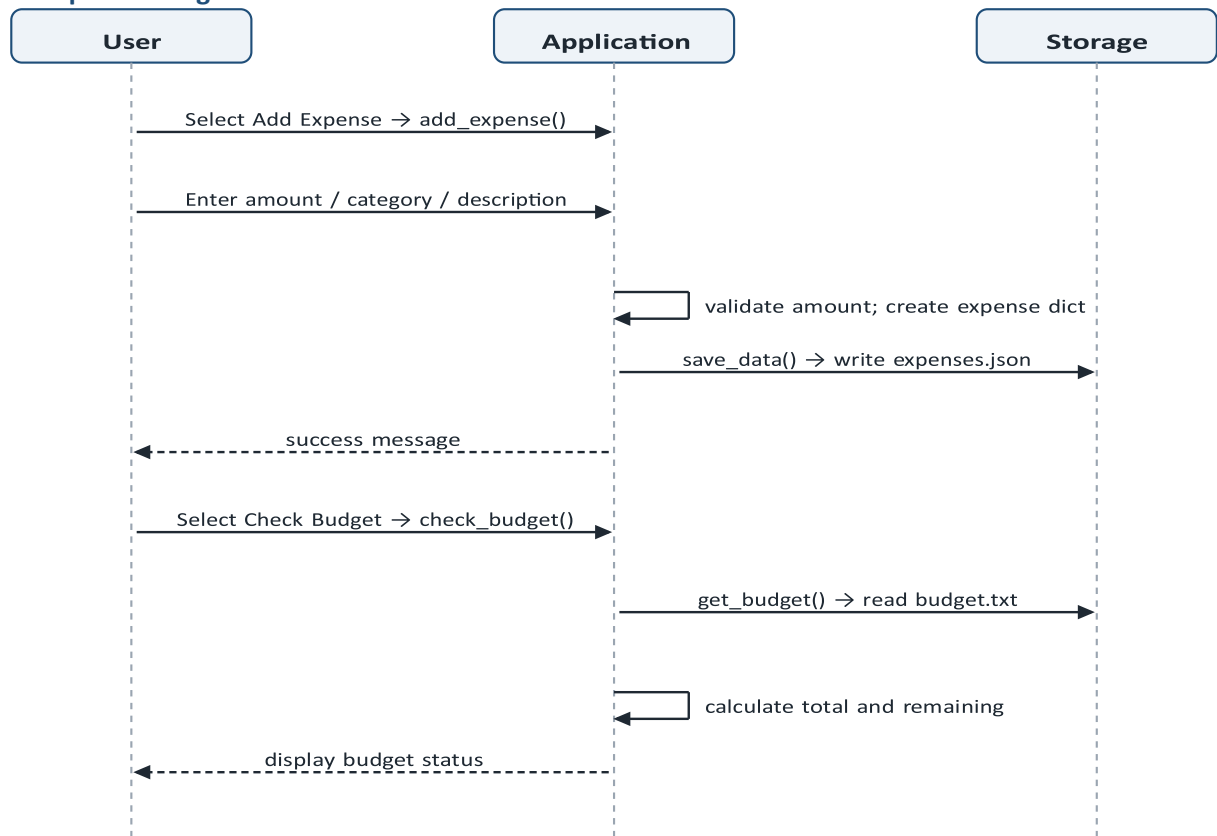


The student is the only external actor. All eight menu operations are initiated by the student.

7.2 Workflow Diagram

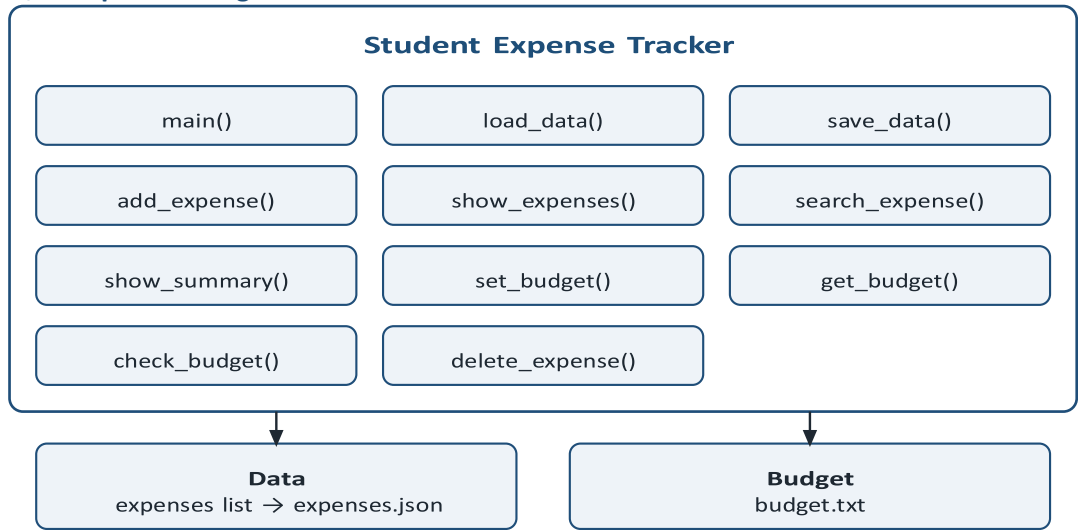


7.3 Sequence Diagram



The sequence shows two representative operations: adding an expense and checking the saved budget.

7.4 Class / Component Diagram



7.5 ER Diagram

Although the project does not use a relational database, its JSON structure can be represented conceptually as an Expense entity.



amount	: float
category	: string
description	: string
date	: string

One Expense record is stored as one dictionary inside the expenses list and serialized to expenses.json.

8. Design Decisions & Rationale

Decision	Rationale
Python standard library	Keeps the project simple and avoids external dependencies.
Functions for each operation	Makes the program easier to read, test and maintain.
List of dictionaries	Provides a straightforward structure for storing multiple expense records.
JSON storage	Human-readable format that is easy to save and load using Python's json module.
Separate budget.txt	Keeps the budget independent from the expense records and simple to update.
Automatic date	Reduces manual input and records the date on which the expense is added.
Exception handling	Prevents invalid numeric input and missing files from immediately crashing the program.

9. Implementation Details

The program begins with an empty global list named **expenses**. The **load_data()** function attempts to read expenses.json and loads the JSON array into this list. If the file does not exist, an empty list is used.

The **add_expense()** function reads an amount, category and description. The current date is generated using `datetime.now()` and formatted as day-month-year. A dictionary is appended to the expenses list and then saved using **save_data()**.

The **show_summary()** function calculates the overall total and maintains a `category_total` dictionary. The **search_expense()** function performs a case-insensitive category comparison. The budget functions use budget.txt to save and retrieve the monthly budget.

The **check_budget()** function computes `remaining = budget - total spending` and reports whether the user is within, exactly at, or above the budget. The **delete_expense()** function removes the selected item using its displayed number and then saves the modified list.

Main menu: The `main()` function runs a while loop and maps choices 1–8 to the corresponding functions. Invalid choices display an error message and return to the menu.

Example of the expense data structure:

```
{"amount": 1000.0, "category": "food", "description": "bistro", "date": "29-09-2026"}
```

10. Screenshots / Results

The screenshots below show the implemented program running successfully. The sample run demonstrates adding expenses, viewing records, setting a budget, checking budget status, searching by category and generating a summary.

Main Menu

```
STUDENT EXPENSE TRACKER
=====
1. Add Expense
2. View Expenses
3. Search Expense
4. Expense Summary
5. Set Monthly Budget
6. Check Budget
7. Delete Expense
8. Exit
=====
Enter your choice: █
```

Add Expense

```
Enter your choice: 1

--- Add Expense ---
Enter amount: 1000
Enter category: food
Enter description: bistro
Expense added successfully!
```

View Expenses

```

--- Your Expenses ---

Expense 1
Amount : 1000.0
Category : food
Description : mayuri
Date : 24-09-2026

Expense 2
Amount : 5000.0
Category : food
Description : mayuri
Date : 24-09-2026

Expense 3
Amount : 10000.0
Category : travel
Description : home return
Date : 28-09-2026

Expense 4
Amount : 10000.0
Category : food
Description : mayuri
Date : 28-09-2026

Expense 5
Amount : 1100.0
Category : food
Description : dinner
Date : 28-09-2026

Expense 6
Amount : 5000.0
Category : travel
Description : home visit
Date : 28-09-2026

Expense 7
Amount : 1000.0
Category : food
Description : bistro
Date : 29-09-2026

```

Search Expense by Category

```

--- Search Expense ---
Enter category: food
₹ 1000.0 | mayuri | 24-09-2026
₹ 5000.0 | mayuri | 24-09-2026
₹ 10000.0 | mayuri | 28-09-2026
₹ 1100.0 | dinner | 28-09-2026
₹ 1000.0 | bistro | 29-09-2026

```

Expense Summary

```

--- Expense Summary ---
Total spending: ₹ 33100.0

Category-wise spending:
food : ₹ 18100.0
travel : ₹ 15000.0

```

Set Monthly Budget

```
Enter your choice: 5

--- Set Budget ---
Enter your monthly budget: 10000
Budget saved successfully!
```

Budget Status

```
--- Budget Status ---
Monthly budget : 10000.0
Total spent    : 33100.0
Remaining      : -23100.0
You have exceeded your budget.
```

Observed Test Result

Metric	Observed value
Monthly budget	₹10,000
Total spending	₹33,100
Remaining budget	-₹23,100
Food spending	₹18,100
Travel spending	₹15,000

11. Testing Approach

Testing was performed by executing the menu options and checking whether the displayed output matched the entered data and calculations.

Test Case	Input / Condition	Expected / Observed Result
Add valid expense	Valid amount, category and description entered	Expense added and saved
Invalid amount	Non-numeric amount	Error message displayed; operation stopped
View expenses	Existing records available	All records displayed
Search category	food	Matching food records displayed
Expense summary	Existing records	Total and category totals calculated
Set budget	10000	Budget saved successfully

Check budget	Budget + saved expenses	Remaining amount and status displayed
Delete expense	Valid expense number	Selected expense removed and data saved
Missing data files	expenses.json / budget.txt absent	Program starts with empty data / asks to set budget

12. Challenges Faced

- **Persistent data:** The program needed a way to retain expenses after closing, so JSON file handling was used.
- **Input validation:** Numeric inputs such as amount and budget can cause errors if text is entered. try/except was added for these cases.
- **Budget calculation:** The application must combine all expense amounts before comparing them with the budget.
- **Category search:** Case-insensitive comparison was used so that inputs such as Food and food can match.
- **Deletion:** The user sees numbered records, so the selected number must be converted to the correct list index using number - 1.

13. Learnings & Key Takeaways

- Learned how to divide a program into reusable functions.
- Understood how lists and dictionaries can represent structured records.
- Practiced reading from and writing to files.
- Learned basic JSON serialization and deserialization.
- Used datetime to generate the current date automatically.
- Applied loops and conditional statements to calculations and menu control.
- Used exception handling to make input processing safer.
- Understood how a small real-world problem can be converted into a working console application.

14. Future Enhancements

- Add edit/update functionality for existing expenses.
- Add date-range filtering and monthly history.
- Add income tracking and balance calculation.
- Add charts for category-wise spending.
- Use SQLite for more scalable structured storage.
- Add a graphical user interface or web interface.
- Add stronger input validation such as preventing negative expenses.
- Support multiple users with separate profiles.

- Export reports to CSV or PDF.

15. References

- Python Standard Library documentation – json module.
- Python Standard Library documentation – datetime module.
- Python documentation – file input/output and exception handling.
- VITyarthi project/report submission instructions provided for the course.

Project files: expense_tracker.py, expenses.json, budget.txt, README.md

Submission note: Replace the cover-page placeholders with your name, registration number, section and faculty details before submitting.